

Python 11

Elias Wachmann

2025

Content

1. Differentiation in Python
 2. Integration in Python
 3. Test Driven Development (TDD) in Python
 4. What is object oriented programming?
 5. Python classes
 6. Numerical Solution of Initial Value Problems
- (BONUS)
7. File I/O (BONUS)
 8. ASCII Encoding (BONUS)
 9. pickle (BONUS)

Differentiation in Python

Numpy `numpy.gradient`

The `numpy.gradient` function calculates the gradient of an array using finite differences. `numpy.gradient`

$$\frac{\partial f}{\partial x} \approx \frac{f(x+h) - f(x-h)}{2h} \quad (1)$$

where h is a small step size. For a one-dimensional array y with spacing dx , the gradient is calculated as:

$$\text{gradient}[i] = \frac{y[i+1] - y[i-1]}{2 \cdot dx} \quad (2)$$

Numpy `numpy.gradient` Example

```
1 import numpy as np
2
3 x = np.array([1, 2, 4, 7, 11])
4 gradient = np.gradient(x)
5 print("Gradient:", gradient)
```

Scipy `scipy.misc.derivative`

The `scipy.misc.derivative` function calculates the numerical derivative of a function.

`scipy.misc.derivative` This function uses the central difference formula: see Equation 1 or Finite difference

```
1 from scipy.misc import derivative
2
3 def f(x):
4     return x**3 + x**2
5
6 deriv = derivative(f, 1.0, dx=1e-6)
7 print("Derivative:", deriv)
```

Integration in Python

Scipy `scipy.integrate.trapezoid`

The `scipy.integrate.trapezoid` function computes the trapezoidal rule to approximate the integral.

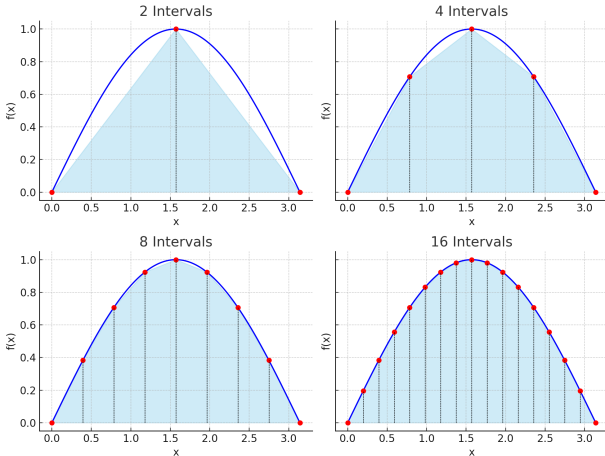
`scipy.trapezoid` It uses the following approximate

formula:

$$\int_a^b f(x) dx \approx \frac{b-a}{2} (f(a) + f(b))$$

Intuition behind the Trapezoid Rule

Increasing Number of Rectangles in the Trapezoid Rule



Scipy `scipy.integrate.trapezoid`

```
1 import numpy as np
2 from scipy.integrate import trapezoid
3
4 x = np.linspace(0, np.pi, 100)
5 y = np.sin(x)
6 integral = trapezoid(y, x)
7 print("Integral using scipy.trapezoid:",
      integral)
```

Self-written Trapezoid Rule

An implementation of the trapezoid rule from scratch.
You have to do that in one of the exercises.

Translate the intuition / formula into maybe a loop or
do it entirely vectorized.

Scipy `scipy.integrate.simpson`

The `scipy.integrate.simpson` function computes the Simpson's rule to approximate the integral.

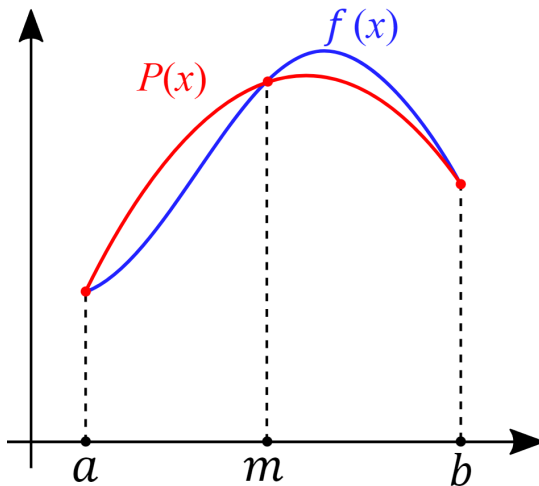
`scipy.simpson` Another way to approximate the

integral is using Simpson's rule:

$$\int_a^b f(x) dx \approx \frac{b-a}{6} \left(f(a) + 4f\left(\frac{a+b}{2}\right) + f(b) \right)$$

Quadratic interpolation is used to approximate the function between the points a and b .

Simpson rule intuition



Scipy `scipy.integrate.simpson`

```
1 import numpy as np
2 from scipy.integrate import simpson
3
4 x = np.linspace(0, np.pi, 100)
5 y = np.sin(x)
6 integral = simpson(y, x)
7 print("Integral using scipy.simpson:",
      integral)
```

Test Driven Development (TDD) in Python

Introduction to TDD

Test Driven Development (TDD) is a software development methodology in which tests are written before the code that needs to be tested.

- Write a test for the new functionality.
- Write the minimum amount of code to make the test pass.
- Refactor the code while ensuring that all tests still pass.

Benefits of TDD

- Ensures code quality and correctness.
- Facilitates better design and architecture.
- Provides documentation for the code.
- Helps in detecting bugs early.

TDD Cycle

The TDD cycle consists of the following steps:

1. **Red:** Write a test that fails.
2. **Green:** Write the minimal code to pass the test.
3. **Refactor:** Improve the code while ensuring tests pass.

Example: Simple Calculator

Let's consider a simple calculator as an example. We want to implement an `add` function.

```
1 def add(a, b):  
2     # Add two numbers and return the result
```

Let us write a test for this function ...

Writing Tests with Pytest

Pytest is a testing framework for Python that makes it easy to write simple and scalable test cases. Install pytest using pip:

```
pip install pytest
```

We specify the test functions using the `test_` prefix. They are automatically detected and run by Pytest.

Pytest Example

Here is an example of a test for the `add` function using Pytest:

```
1 from calculator import add
2
3 def test_add():
4     assert add(1, 2) == 3
5     assert add(-1, 1) == 0
6     assert add(-1, -1) == -2
```

Place this test in a file prefixed with `test_` (e.g., `test_calculator.py`) into the same directory as the code.

Running Tests

To run the tests, simply execute the following command in the terminal:

```
pytest
```

Pytest will automatically discover and run all the test files that match the pattern `test_*.py`.

Refactoring

After making the test pass, you can refactor the code to improve its quality. Ensure that all tests still pass after refactoring.

```
1 def add(a, b):  
2     return a + b  
3  
4 def subtract(a, b):  
5     return a - b
```

Refactoring

```
1 from calculator_refactored import add,  
    subtract  
2  
3 def test_add():  
4     assert add(1, 2) == 3  
5     assert add(-1, 1) == 0  
6     assert add(-1, -1) == -2  
7  
8 def test_subtract():  
9     assert subtract(2, 1) == 1  
10    assert subtract(1, 1) == 0  
11    assert subtract(0, 1) == -1
```


Best Practices

- Write small, focused tests.
- Use descriptive test names.
- Keep tests independent.
- Regularly run all tests.
- Refactor tests along with the code.

Conclusion

Test Driven Development (TDD) helps in ensuring **code quality**, detecting bugs early, and improving the design and architecture of the code.

Pytest is a powerful and easy-to-use testing framework for implementing TDD in Python.

Start with small steps, and iteratively build a robust suite of tests alongside your code.

What is object oriented programming?

Intro to object oriented programming

How do we classify things in the real world?

We group them by their properties and their behavior.

An **object** like a planet can be described by its properties like mass, radius, position, velocity, etc.

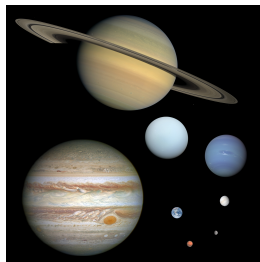
We can also describe its behavior, like how it moves around the sun.

What are classes?

Classes are a way to abstract real world things into categories.

Not all planets are the same, but they can be described by the same properties and behavior.

We can use a **class** that describes the properties and behavior of a planet (an **object**).



https://upload.wikimedia.org/wikipedia/commons/c/cf/Planet_collage_to_scale.jpg

Describe objects within a class

How can we distinguish between different planets now?

Earth: $m = 5.97 \times 10^{24} \text{ kg}$, $r = 6371 \text{ km}$, ...

Mars: $m = 6.41 \times 10^{23} \text{ kg}$, $r = 3389.5 \text{ km}$, ...

Jupiter: $m = 1.90 \times 10^{27} \text{ kg}$, $r = 69911 \text{ km}$, ...

Python classes

Let's define a Planet class

In python we can define a class like this:

```
1 class Planet():
2     def __init__(self, name_in, radius_in,
3         mass_in):
4         self.name = name_in
5         self.radius = radius_in
6         self.mass = mass_in
```

The **class**-keyword tells python that we want to define a class.

What is the `__init__` function?

The `__init__` function is called when we create a new object of the class.

Objects are therefore concrete **instances** of a class.

In our example the Planet class takes the arguments **mass** and **radius** and stores them as **attributes** of the object.

How can we access these attributes and what is the **self**-keyword?

Idea behind self keyword

The **self** keyword is a reference to the object itself.

What does this mean?

2 options:

- Just accept it as is and use it.
- Understand what it does.

When we create an object of a class, we can access its attributes and methods using the **self** keyword inside the class.

Python's self keyword

The **self** keyword is a reference to the object itself.

```
1  def __init__(self, name_in, radius_in,
2      mass_in):
3      self.name = name_in
4      self.radius = radius_in
5      self.mass = mass_in
```

This means: If you want to create an attribute of a class, you simply write `self.variable_name`

self also has to be passed in as the first argument to every function defined inside the class (for context)!

Methods – class functions

Methods are functions inside a class. They can only be accessed by objects of that class and **self** is their first argument.

```
1 class Planet():
2     def __init__(self, name_in, radius_in,
3         mass_in):
4         self.name = name_in
5         self.radius = radius_in
6         self.mass = mass_in
7
8     def getData(self):
9         return {"name": self.name, "radius":
10             self.radius, "mass": self.mass}
```

Let's create a Planet

Finally let's create a planet object:

```
1 earth = Planet("Earth", 6371, 5.972e24)
2 mars = Planet("Mars", 3389, 6.39e23)
```

Just pass in the initial arguments just like you would for functions.

```
1 earth_data = earth.getData()
2 print(earth_data["name"]) # Earth
```

We can now call our defined class method on objects.

Modify objects

Certainly a method can modify attributes of an object like in the following example the name.

```
1     def changeName(self, newname):  
2         self.name = newname  
  
1 earth.changeName("Earth2.0")  
2 earth_data = earth.getData()  
3 print(earth_data["name"])    # Earth2.0
```

This only changes the name of the ,earth'-object while the ,mars'-object is not affected.

Numerical Solution of Initial Value Problems (BONUS)

Mathematical Introduction

For initial value problems, we discretize the time using $t_n = t_0 + n\Delta t$ and introduce the notation $y_n = y(t_n)$. Thus, we write

$$\dot{y}_n = f(t_n, y_n)$$
$$y_{n+1} = y_n + \int_{t_n}^{t_{n+1}} dt' f(t', y(t'))$$

which is still exact. We can now approximate the integral in various ways.

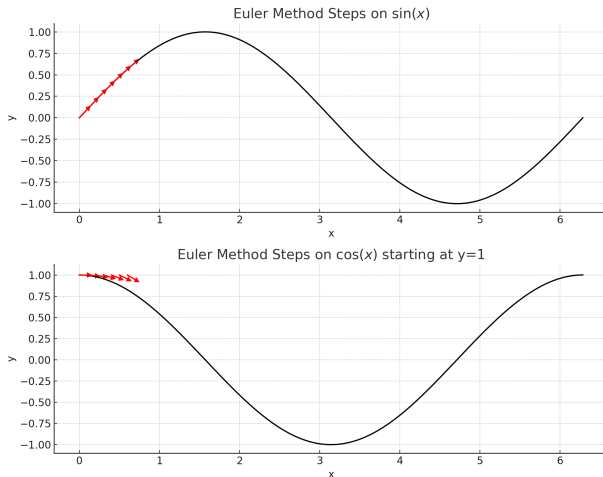
Explicit Euler Method

The explicit Euler method is the simplest method for numerically solving initial value problems. It uses a simple forward difference to approximate the next solution:

$$y_{n+1} = y_n + \Delta t \cdot f(t_n, y_n)$$

- **Intuition:** We take a small step Δt along the tangent to the solution curve.
- **Accuracy:** This method is only accurate for small Δt , as it does not account for the curvature of the solution curve.

Explicit Euler Method intuition



Problems with the Explicit Euler Method

As can be seen in the figure on the last slide the explicit Euler method can lead to large errors.

Given a start at an extreme point of the function, the values will stay put due to the tangent being parallel to the x-axis:

$$y_{n+1} = y_n + \Delta t \cdot f(t_n, y_n) = y_n$$

because the derivative $f(t_n, y_n) = 0$.

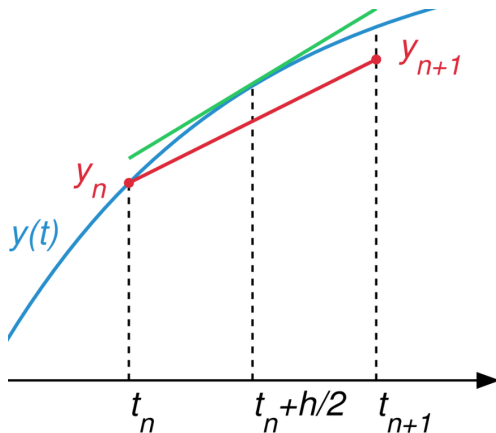
Midpoint Method

The midpoint method improves accuracy by using the derivative at the midpoint of the interval:

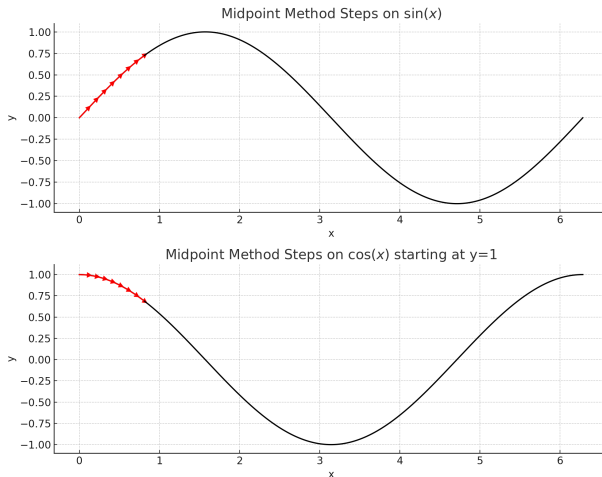
$$y_{n+1} = y_n + \Delta t \cdot f \left(t_n + \frac{\Delta t}{2}, y_n + \frac{\Delta t}{2} \cdot f(t_n, y_n) \right)$$

- **Intuition:** We first compute a preliminary value at the midpoint of the interval and then use this value to calculate the next step.
- **Accuracy:** Higher than the explicit Euler method, as it better accounts for the curvature.

Midpoint Method intuition



Midpoint Method intuition



Improvement of the Midpoint Method over the Explicit Euler Method

Even if the derivative is zero: $f(t_n, y_n) = 0$, the midpoint method will still take a step.

$$\begin{aligned} y_{n+1} &= y_n + \Delta t \cdot f \left(t_n + \frac{\Delta t}{2}, y_n + \frac{\Delta t}{2} \cdot f(t_n, y_n) \right) \\ &= y_n + \Delta t \cdot f \left(t_n + \frac{\Delta t}{2}, y_n \right) \\ &= y_n + \Delta t \cdot f \left(t_n + \frac{\Delta t}{2}, y_n \right) \neq y_n \end{aligned}$$

File I/O (BONUS)

File I/O – Basics

Python uses with statements to open and close files.

```
1 with open("slides/08/examples/testfile1.txt"  
    , "w") as f:  
2     f.write("Hello, world!\n")  
3     f.write("This is a test file.\n")  
4  
5 with open("slides/08/examples/testfile1.txt"  
    , "r") as f:  
6     for line in f:  
7         print(line, end=" ")
```

File I/O – file modes

The `open` function can be used to open files in different modes (multiple are possible):

File Mode	Description
r	Read mode
w	Write mode
x	Exclusive creation mode
a	Append mode
b	Binary mode
t	Text mode
+	Read and write mode

File I/O – Reading Files

To read the contents of a file, you can use the `read()` method of the file object. This method reads the entire contents of the file and returns it as a string.

```
1 with open('slides/08/examples/input.txt', 'r') as f:
2     text = f.read()
3     print(text)
```

In this example, we open the file `input.txt` in read mode (`r`), read its contents using the `read()` method, and print the contents to the console.

File I/O – Writing Files

To write to a file, you can use the write() method of the file object. This method writes the specified string to the file.

```
1 with open('slides/08/examples/input.txt', 'a') as f:
2     f.write("This is a test file.\n")
3     f.writelines(["Boring text 1.\n", "Boring text 2.\n"])
```

In this example, we open the file `example.txt` in write mode (`w`), write some strings to the file using the write() method.

File I/O – Reading multiple lines

To read multiple lines from a file, you can use the `readlines()` method of the file object. This method reads the entire contents of the file, and returns each line as an item in a list.

```
1 with open('slides/08/examples/input.txt', 'r') as f:
2     for line in f.readlines():
3         print(line, end=" ")
```

You do not need to close the file when using the `with` statement. It is automatically closed when the `with` block is exited.

File I/O – Binary files

To read or write binary files, you can use the rb and wb modes, respectively:

```
1 with open('data.bin', 'wb') as f:
2     f.write(b'\x00\x01\x02\x03')
3
4 with open('data.bin', 'rb') as f:
5     data = f.read()
6     print(f"Original data: {data}")
```

But how do I know what is in there ... ?

File I/O – Encoding

There are many encodings for text files.

- ASCII: 7-bit encoding, 128 characters
- Unicode: 16-bit encoding, 65536 characters
- UTF-8: 8-bit encoding, variable length, ASCII compatible
- ISO-8859-1: 8-bit encoding, 256 characters (Latin-1)

To interpret a file in the right way, you have to specify the encoding.

File I/O – Encoding

Specify the encoding when opening the file using the encoding argument:

```
1 with open("slides/08/examples/testfile2.txt"  
    , "w", encoding="UTF-8") as f:  
2     f.write("Hello, Österreich!\n")  
3  
4 with open("slides/08/examples/testfile2.txt"  
    , "r", encoding="ASCII") as f:  
5     data = f.read()  
6     print(f"Original data: {data}")
```

This will fail! Why?

File I/O – Encoding

The specified encoding has to match the encoding of the file!

The file was written using UTF-8 encoding and later read using ASCII encoding which is doomed to fail, since an ‚Ö‘ is in the textfile.

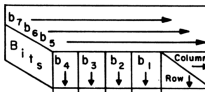
```
Traceback (most recent call last):
  File "/home/etschgi1/Desktop/REPOS/exercises-python/slides/08/examples/fileio5.py", line 5, in <module>
    e>
    data = f.read()
  File "/usr/lib/python3.10/encodings/ascii.py", line 26, in decode
    return codecs.ascii_decode(input, self.errors)[0]
UnicodeDecodeError: 'ascii' codec can't decode byte 0xc3 in position 7: ordinal not in range(128)
```

ASCII Encoding (BONUS)

ASCII - Table

ASCII (American Standard Code for Information Interchange) is a character encoding standard that assigns unique numeric codes to each character in the English alphabet, as well as various punctuation marks and other symbols.

ASCII - Table



Column	0	1	2	3	4	5	6	7
0	NUL	DLE	SP	0	@	P	`	p
1	SOH	DC1	!	1	A	Q	a	q
2	STX	DC2	"	2	B	R	b	r
3	ETX	DC3	#	3	C	S	c	s
4	EOT	DC4	\$	4	D	T	d	t
5	ENQ	NAK	%	5	E	U	e	u
6	ACK	SYN	&	6	F	V	f	v
7	BEL	ETB	'	7	G	W	g	w
8	BS	CAN	(	8	H	X	h	x
9	HT	EM	)	9	I	Y	i	y
10	LF	SUB	*	:	J	Z	j	z
11	VT	ESC	+	;	K	[	k	{
12	FF	FS	,	<	L	\	l	
13	CR	GS	-	=	M	]	m	}
14	SO	RS	.	>	N	^	n	~
15	SI	US	/	?	O	_	o	DEL

https://de.wikipedia.org/wiki/American_Standard_Code_for_Information_Interchange#/media/Datei:USASCII_code_chart.png

ASCII - Table

This way everything is up to interpretation: You can read the file as ASCII if there are no special characters in it.:

```
1 with open('slides/08/examples/ascii.txt', '
  wb') as f:
2     f.write(b'\x41\x42\x43')
3
4 with open('slides/08/examples/ascii.txt', 'r
  ', encoding='ascii') as f:
5     print(f.read()) # ABC
```

Using other encodings

With another encoding you can read special characters such as ,Ö‘:

```
1
2 with open('slides/08/examples/ascii.txt', '
   wb') as f:
3     f.write(b'\x41\x42\x43\xD6')
4
5 with open('slides/08/examples/ascii.txt', 'r
   ', encoding='ISO-8859-1') as f:
6     print(f.read()) # ABCÖ
```

pickle (BONUS)

`pickle`

`pickle` is a module that allows you to store almost any Python object (lists, dictionaries, ...) in a file.

`pickle` is a binary format, so you have to open the file in binary mode (`wb` or `rb`).

Use the `dump()` method to write an object to a file, and the `load()` method to read an object from a file.

pickle – Example

```
1 import pickle
2
3 # Create a dictionary
4 dic_ = {'a': 1, 'b': 2, 'c': 3}
5 with open('slides/08/examples/dic_.pkl', 'wb') as f:
6     pickle.dump(dic_, f)
7 with open('slides/08/examples/dic_.pkl', 'rb') as f:
8     data = pickle.load(f)
9     print(f"Original data: {data}")
```